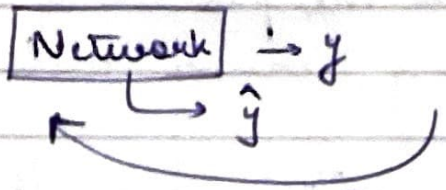
DL GenAI - Week-5

L1 Optimizaⁿ in DL - Challenges



- Reference → d2l.ai
- optimizaⁿ algos to train our DNN.
  → updates the model params iteratively.
  → goal is to minimize the loss value on the training dataset.
- why more understanding is reqd. for us?
  1. training efficiency → performance of optimizaⁿ algo & model's training time. You have to save computational time & resources. Be training prepared for adaptive learning.
  2. model performance → HPT is reqd. in a targeted manner.
- goal of optimizaⁿ → minimize the objective funcⁿ (training error / empirical risk)
  ↳ our focus is to perform well on unseen data (minimize the generalizaⁿ error - true risk)
  ↳ this is the final goal.
- True Risk → the expected loss on the entire popⁿ of data. This is smooth. We want to find its true minimum.

__/__/___

- Empirical Risk - The avg. loss on our finite training dataset. It is often noisy & less smooth. This what we minimize.
- Optimiza^n → The process of reducing the training error with algos. like SGD, adam.
- Regularia^n → Techniques that control model complexity & reduces generaliza^n error.
- Almost all optimiza^n problems in DL are non-convex (messy) → extremely complex. like a rugged mountain range.
- 3 major challenges -
  1. local minima → getting stuck in a some valley instead of lowest pt.
  2. saddle pt. → getting stuck on a flat pt. that is min. in 1 direc^n but max in another.
  3. vanishing gradient → gradient so small that learning stops effectively
     ↳ mostly machines have limited precision.

- Local Minima
  → A pt. x where $f(x)$ is the minimum value over any other pts. in its immediate vicinity.
  → Global minima → x where $f(x)$ is lowest

__/__/__

- Saddle Pt.
  → where all gradients of a func^n vanish ($\nabla f(x) = 0$) but is neither local minima or local maxima.
- Hessian matrix → matrix of second derivatives
  ↳ local minima → all eigenvalues of Hessian is positive.
  ↳ saddle pt. → some λ_is are +, some are -.
  → at the zero-gradient pt., the type of depends on this Hessian matrix.

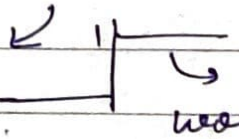
Eg. $f(x, y) = x^2 - y^2$ at $(0,0)$ → gradient is $(0,0)$
minima wrt x.
maxima wrt y.
In 2D, looks the shape of a saddle.

- Vanishing gradient
  → when gradient become extremely small, causing the updates to be effectively negligible. Thus, learning stops.
  → you have to play with activa^n func^n.

Eg. $f(x) = \tanh(x)$, $f'(x) = 1 - \tanh^2(x)$

$x = 4$
$f'(x) = 0.0013$ so small

-that's why we use ReLu.

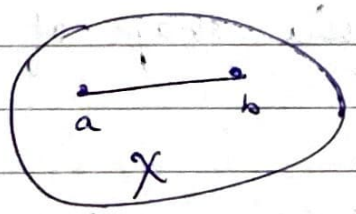


↳ when they work, they work very good.

∴ that's why used in CNNs.

L2: Convex Optimization

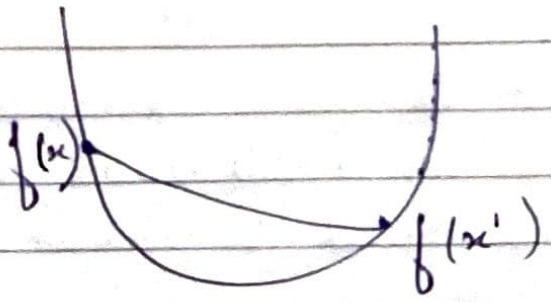
- Reasons to know convexity —
  1. analytical tractability → easier to understand
  2. algo. design & testing → works as a sanity check for convex optima
  3. Local properties → even in case of non-convex algorithms, local properties are convex.
- A set $\mathcal{X}$ is a vector space is convex if for any 2 pts. $a, b \in \mathcal{X}$, the entire line segment connecting a & b is also in $\mathcal{X}$.



for all $\lambda \in [0,1]$, we have
$\lambda a + (1-\lambda) b \in \mathcal{X}$
when $a, b \in \mathcal{X}$

- Properties —
  1. intersection of 2 (or more) convex sets is also convex
  2. not necessarily true for unions.
- Given a convex set $\mathcal{X}$, if func^n $f: \mathcal{X} \to \mathbb{R}$ is convex if for all $x, x' \in \mathcal{X}$ & for all $\lambda \in [0,1]$

$$\lambda f(x) + (1-\lambda) f(x') \geq f(\lambda x + (1-\lambda) x')$$



Eg. $0.5x^2$, $e^{0.5x}$ they are convex.
but $\cos(\pi x)$ is not convex.

→ Any local minima is also the global minima, for a convex func^n f.

- The second derivative test
  → $f: R \to R$; f is convex iff. if its second derivative is non-negative $f''(x) \geq 0, \forall x$.
  → In multidim, f is convex iff. Hessian matrix, $H = \nabla^2 f(x)$ is positive semidefinite for all x.
- Jenson's Inequality
  → for a convex func^n f, non-negative wts. α_i s.t. $\sum \alpha_i = 1$ & a RV x.

$$\sum_i \alpha_i f(x_i) \geq f\left(\sum_i \alpha_i x_i\right)$$

or

$$E[f(X)] \geq f(E[X])$$

- Convex optimizat^n gives good tools to handle constraints efficiently.

Eg. $\min_x \; f(x)$

subj. to $c_i(x) \leq 0 \quad i = 1 \dots n$

f & c_i are convex functions.

__/__/____

1. Lagrangian → converts constrained into unconstrained.
   ↳ introduces new variables $\alpha_i \geq 0$ called lagrange multipliers.

$$L(x, \alpha_i \dots \alpha_n) = f(x) + \sum_{i=1}^{n} \alpha_i c_i(x)$$

Eg: for finding a saddle pt. of L.

$$\max_{\alpha_i \geq 0} \min_{x} L(x, \alpha)$$

2. Penalty → we add a penalty term to the objective.

$$\min f(x) + \sum_{i=1}^{n} \alpha_i c_i(x)$$

This is an approximate way to satisfy constraints.

3. Projections → take an optimizn step & then project the result back to the feasible set.

$$\text{Proj}_{\chi}(x) \doteq \underset{x' \in \chi}{\text{argmin}} \; \|x - x'\|$$

This finds the closest pt. in the convex set χ to the pt. x.

- so, remember, we can never get stuck in the local minima valley.

L3. Gradient descent Algo.

- Taylor Expansion → for small displacement ε, we can approximate using 1st order taylor expansion.

$$f(x+\varepsilon) \approx f(x) + \varepsilon^T \nabla f(x)$$

(func" at new pt. ← ; $f(x)$ → func" at old pt.; ε → rate of Δ (step size))

we move in the direc" where steepness decreases the most, we make small Δs in that direc".

$$\nabla f(x) = \nabla f \cdot \varepsilon \qquad \vec{\nabla} f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

→ to make $f(x+\varepsilon)$ as small as possible, we need to make $\varepsilon^T \nabla f(x)$ as negative as possible. We try to minimize $\varepsilon^T \nabla f(x)$.

Δf is min, when $\cos\theta = -1$, $\theta = 180°$

$\vec{\Delta x} \longleftarrow \bullet \longrightarrow \nabla f$

→ maximum decrease the value in the direc" opp. to the gradient

$$\boxed{\varepsilon = -\eta \nabla f(x)}$$

$\eta > 0$, small +ve scalar.

↓ learning rate

- GD Theorem → start with initial value x_0, iteratively update the params.

$$x_{t+1} \leftarrow x_t - \eta \nabla f(x_t)$$

we repeat it until a stopping condi^n

Eg. $f(x) = x^2$ obj. $\Rightarrow x^2$

$f'(x) = 2x$

Update rule $\Rightarrow$ $x_{t+1} \leftarrow x_t - \eta(2x_t)$

start, $x = 10$, with $\eta = 0.2$

$x_1 = x_0 - \eta(2x_t)$

$= 10 - 0.2(20) = 10 - 4 = 6$

$x_2 = 6 - 0.2(12) = 6 - 2.4 = 3.6$

⋮

progress is very slow, with $\eta = 0.2$

if η is too large, then, it will overshoot the minima.

→ HPT → expt. to get the best result.

Eg. $f(x) = x_1^2 + 2x_2^2$

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 \\ 4x_2 \end{bmatrix}$$

difficult to get 1 single good η.

- gradient use 1st order info. Can we use second order info?

⇐ Newton's method

$$f(x+\varepsilon) \approx f(x) + \varepsilon^T \nabla f(x) + \frac{1}{2}\varepsilon^T H \varepsilon$$

↳ this is expensive $O(n^3)$

$H = \nabla^2 f(x)$

↳ hessian matrix of second derivatives

↳ using second-order taylor expansion.

$$\nabla f(x) + H\varepsilon = 0$$

$$\Rightarrow \boxed{\varepsilon = -H^{-1} \nabla f(x)}$$

← to find the minima of this quadratic approxima"

→ this helps in fast convergence. For quadratic ones, it converges in exactly 1 step.

→ if func" is not convex, H has -ve eigen-values. It starts to move towards maxima.

- Batch GD (Gradient descent)

↳ the obj. func" is an avg. over a training dataset of n egs. $\boxed{f(x) = \frac{1}{n}\sum_{i=1}^{n} f_i(x)}$

→ the true or batch gradient is also an avg. (1 epoch)

→ to get this, you have to do a full pass over the entire training dataset.

→ 1 single gradient update is $O(n)$. We shouldn't do this.

- SGD (Stochastic gradient descent)
  → instead of calculating the gradient over the entire dataset, we just take 1 random eg. & compute the gradient for it.
  → at each iter^n t →
  1. uniformly sample an index i
  2. compute SGD → $\nabla f_{i_t}(x_i)$
  3. update →

$$x_{t+1} \leftarrow x_t - \eta \nabla f_{i_t}(x_t)$$

  → key advantages:
  1. cost: $O(n) \rightarrow O(1)$
  2. unbiased estimate

- There are some key obs.
  → GD path → smooth deterministic, directly towards min.
  → SGD path → noisy, stochastic & rough estimates
  ↳ we have to use some η scheduler.

- so, ∴, we can use Minibatch SGD → we use small batch of eg's. It is a std. for training the deep neural networks.

__/__/____

- why mini batch SGD works?
  - → SGD → element-by-element
  - → Mini Batch SGD → block by block
  - → GD → full matrix

this can be handled by today's GPUs → computational efficiency.

uses vectorization

- Mini batch SGD
  1. $B_t \subset \{1, \ldots, n\}$ of size b
  2. $\nabla f_{B_t}(x_t) = \frac{1}{b} \sum_{i \in B_t} \nabla f_i(x_t)$
  3. $x_{t+1} \leftarrow x_t - \eta \nabla f_{B_t}(x_t)$

  → computational & statistical efficiency.

L4 Accelerating Gradient Descent

- why we schedule η?
  - → magnitude → η too large, it diverges; Too small, it converges slowly.
  - → decay → η should decay over time.
  - → warmup → random initialization so starting steps are slow.
- common schedules
  - → piecewise const. → $\eta(t) = \eta_i$ if $t_i \leq t < t_{i+1}$

→ exponential decay ⇒ $\eta(t) = \eta_0 \cdot e^{-\alpha t}$

→ polynomial ⇒ $\eta(t) = \eta_0 (\beta t + 1)^{-\alpha}$

→ inverse time ⇒ $\eta(t) = \frac{\eta_0}{(1+kt)}$

→ cosine annealing ⇒

$$\eta(t) = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})\left(1 + \cos\left(\frac{t}{T_{max}}\pi\right)\right)$$

- GD is currently struggling –
  → gradient sometime much larger in some direc^n
  → $\eta \uparrow$, too much divergence for some var.
  → $\eta \downarrow$; painfully slow convergence.
- Momentum
  → down the hill, we get momentum, oscillat^{ns} tend to cancel out.
  → if flat reg., it consistent acc.

vanilla GD ⇒ $w \leftarrow w - \eta v$ $\quad \vec{v} = \nabla J$.

$g_t = \nabla f(x_t)$ → prev. velocity

$v_t \leftarrow \beta v_{t-1} + g_t$ → current gradient

this accumulates the velocity

$x_{t+1} \leftarrow x_t - \eta v_t$

↳ update with velocity

v_t, can be written as → exponentially gives imp. to recent gradients.

$$v_t = \sum_{\tau=0}^{t} \beta^{t-\tau} g_\tau$$

- Adagrad (Adaptive Gradients)
→ η over time ↓ for all params. Eg. LLM train, common words eg. frequent updates, while rare words recieve very few updates.
→ ∴, we need a mechanism of per-parameter learning rate. 1st case should have higher η & 2nd case should have lower η.
→ Adagrad adapts the η for each param. based on the history of its gradients.

element-wise multiplication

$$s_t \leftarrow s_{t+1} + g_t \odot g_t$$ accumulates squared gradient, element wise

$$x_{t+1} \leftarrow x_t - \frac{\eta}{\sqrt{s_t + \varepsilon}} \odot g_t$$ update with scale gradient

→ leads to automatic scheduling

→ it acts as a pre-conditioner
→ the accumulator s_t ↑. The η for all params will approach 0, stopping training prematurely.

- RMS Prop.
→ for adagrad, sq. gradients are stored over entire history
→ in RMS Prop, we use exponentially weighted moving avg, & window.

$$s_t \leftarrow \gamma s_{t-1} + (1-\gamma)(g_t \odot g_t)$$

$$x_{t+1} \leftarrow x_t - \frac{\eta}{\sqrt{s_t + \varepsilon}} \odot g_t$$

RMSProp ⇒ Adagrad + "forgetting" mechanism

- Adadelta
  → remove global η. Adapts η with past parameter updates.
  → 2 variables → s_t, Δx_t
  → algo. (ρ → decay rate)
  1. $s_t \leftarrow \rho s_{t-1} + (1-\rho)(g_t \odot g_t)$
  2. $g_t' = \frac{\sqrt{\Delta x_{t-1} + \varepsilon}}{\sqrt{s_t + \varepsilon}} \odot g_t$ → this works as η.
  3. $\Delta x_t \leftarrow \rho \Delta x_{t-1} + (1-\rho)(g_t' \odot g_t')$
  4. $x_{t+1} \leftarrow x_t - g_t'$

- Adam (Momentum + RMS Prop)
  → keep track of past gradients and MA of past squared gradients.
  → algo.
  1. $v_t \leftarrow \beta_1 v_{t-1} + (1-\beta_1) g_t$
  2. $s_t \leftarrow \beta_2 s_{t-1} + (1-\beta_2)(g_t \odot g_t)$
  3. correct the bias

$$\hat{v}_t = \frac{v_t}{1-\beta_1^t} \qquad \hat{s}_t = \frac{s_t}{1-\beta_2^t}$$

4. $x_{t+1} \leftarrow x_t - \eta \frac{\hat{v}_t}{\sqrt{\hat{s}_t + \varepsilon}}$

→ $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 0.001$ (common values)

L5 GD demo

- code walkthroughs → on small datasets, not very realistic.
- we are applying on Linear Regression on a synthetic dataset → $N\sim(y, std.)$

$y = wx + b + noise$

- x → feature
- w, b → parameters

→ expectation of the algo. is the value of w & b.
→ after that we will try to improve it.

$$w = w - \eta \frac{\partial J}{\partial w}$$

$$b = b - \eta \frac{\partial J}{\partial w}$$

→ we will try to solve this

$$J^i = (\hat{y}^i - y^i)^2 \rightarrow \hat{y}(b)$$

$$\frac{\partial J^i}{\partial b} = \frac{\partial J^i}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial b} = 2(\hat{y}^i - y^i)$$

$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum_{i=1}^{n} (\hat{y}^i - y^i)$$

$$\frac{\partial J}{\partial w} = \frac{2}{n} \sum_{i=1}^{n} (\hat{y}^i - y^i) x^i$$

- generate the synthetic dataset for LR.
- plot $\hat{y}$, y & residuals.
- defina^n to compute the loss

$$\frac{\sum (y - \text{predictions})^2}{n}$$

- defina^n to compute the gradients
- define the algorithm → BGD (Batch Gradient Descent)
  - ↳ algorithm → SGD, mini-batch SGD
- these results are not final, we have to HPT, then, you will get good results.
  - ↳ (HPT optimisa^n)
- batch-size also becomes the HP.

LC Accelerating GD - demo.

- momentum, RMSProp, Adagrad, Adam
  - ↳ we will experiment using these techniques.

- defina" of vanilla gradient descent.

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta J(\theta_t) \rightarrow \text{update rule}$$

- define the momentum-based optimiza":
- NAG → Nesterove Accelerated Gradient

$$v_t = \beta v_{t-1} + \nabla_\theta J(\theta_t - \alpha \beta v_{t-1})$$

$$\theta_{t+1} = \theta_t - \alpha v_t$$

helps in smoothening, it anticipates where it is going.

- define the adaptive learning rate methods
  → adagrad
  → RMSProp (Root Mean Square Propagation)
  → Adadelta
- finally, define the Adam family of optimizers.
  → Adam (Adaptive Moment Estimation)
  → AdamW (Adam with decoupled weight decay)
  ↳ L2 Regularization

$$g_t = \nabla_\theta J(\theta_t) + \lambda \theta_t$$

- finally test these optimizers & plot these functions on the toy problems

Adam ≥ RMSProp > Momentum/NAG > Adadelta > AdaGrad

Vanilla GD

General Hierarchy